

Analysis Report Template

Student Full Name	Kanumul Deniya Liyanage Sanidu Harsha	Student ID No.	W2149539
-------------------	---------------------------------------	----------------	----------

Case Study Title
Case Study (A): Predicting Clients Loan Status. [Total 43 Marks] Research Question: Does machine learning have the potential to assist banker and finance analysts in predicting which client can be approved a loan?

Task (1) – Domain Understanding: Classification

The financial risk analysts decided that classification modelling is required. Indicate in the table below for each of the listed variables in your data which ones you should RETAIN and can be included in the classification modelling of loan approval status, should DROP (REMOVE). Justify your decision logically and/or by research (for any research, include in-text citation using the Harvard style) [3 Marks]		
Variable Name	RETAIN or DROP	Brief justification for retention or dropping with in-text citation.
ID	DROP	This variable is a unique identifier and does not contribute to prediction, therefore it has no predictive value
Sex	RETAIN	Gender may influence financial behaviour and lending decisions, making it relevant for prediction
Age	RETAIN	Age reflects financial maturity and stability, which can affect loan repayment ability
Education Qualifications	RETAIN	Education level is often associated with income level and employment stability
Income	RETAIN	Income is a key indicator of a client’s ability to repay the loan
Home Ownership	RETAIN	Property ownership reflects financial strength and creditworthiness
Employment Length	RETAIN	Longer employment indicates stable income and lower risk
Loan Intent	RETAIN	The purpose of the loan can influence the likelihood of repayment
Loan Amount	RETAIN	Loan size directly impacts repayment burden and approval decisions
Loan Interest Rate	RETAIN	Interest rate reflects the risk level assigned to a borrower
Loan-to-Income Ratio (LTI)	RETAIN	This ratio is a critical indicator of financial burden and repayment capacity
Payment Default on File	RETAIN	Past defaults are strong predictors of future credit risk
Credit History Length	RETAIN	Longer credit history provides better insight into borrower reliability

Loan Approval Status	RETAIN	This is the dependent variable to be predicted in the classification model
Maximum Loan Amount	DROP	This variable relates to a different prediction task (regression), not classification.
Credit Application Acceptance	DROP	This duplicates the target concept and may cause data leakage, so it should be excluded.
References		
Han, J., Kamber, M. and Pei, J. (2011) Data Mining: Concepts and Techniques. 3rd ed. Elsevier. Khandani, A.E., Kim, A.J. and Lo, A.W. (2010) ‘Consumer credit-risk models via machine-learning algorithms’, Journal of Banking & Finance, 34(11), pp. 2767–2787. Thomas, L.C., Edelman, D.B. and Crook, J.N. (2017) Credit Scoring and Its Applications. 2nd ed. SIAM.		

Task (2) – Exploring and Understanding Your Dataset

1) basic descriptive stats

Basic descriptive statistics were generated to understand the distribution of numerical variables such as age, income, and loan amount.

	id	age	income	employment_length	loan_amount	loan_interest_rate	loan_income_ratio	credit_history_length	loan_approval_status	max_allowed_loan
count	58645.000000	58639.000000	5.864500e+04	58645.000000	58645.000000	58634.000000	58645.000000	58645.000000	58645.000000	5.864500e+04
mean	29322.000000	27.550913	6.404617e+04	4.703487	9217.556518	10.677526	0.159238	5.813556	0.142382	6.975472e+04
std	16929.497605	6.033217	3.793111e+04	4.004982	5563.807384	3.036034	0.091692	4.029196	0.349445	6.175091e+04
min	0.000000	20.000000	4.200000e+03	0.000000	500.000000	-11.140000	0.000000	2.000000	0.000000	-2.426900e+06
25%	14661.000000	23.000000	4.200000e+04	2.000000	5000.000000	7.880000	0.090000	3.000000	0.000000	3.800300e+04
50%	29322.000000	26.000000	5.800000e+04	4.000000	8000.000000	10.750000	0.140000	4.000000	0.000000	6.239200e+04
75%	43983.000000	30.000000	7.560000e+04	7.000000	12000.000000	12.990000	0.210000	8.000000	0.000000	9.271600e+04
max	58644.000000	123.000000	1.900000e+06	150.000000	35000.000000	23.220000	0.830000	30.000000	1.000000	2.638778e+06

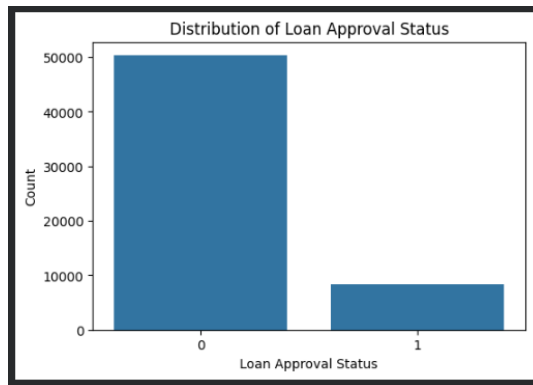
2) variable scale type

The dataset contains both numerical variables (integer and float) and categorical variables (object type).

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 58645 entries, 0 to 58644
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   id                  58645 non-null   int64
1   age                 58639 non-null   float64
2   income              58645 non-null   int64
3   home_ownership      58645 non-null   object
4   employment_length   58645 non-null   int64
5   loan_intent          58645 non-null   object
6   loan_amount         58645 non-null   int64
7   loan_interest_rate   58634 non-null   float64
8   loan_income_ratio    58645 non-null   float64
9   payment_default_on_file 58640 non-null   object
10  credit_history_length 58645 non-null   int64
11  loan_approval_status  58645 non-null   int64
12  max_allowed_loan     58645 non-null   int64
dtypes: float64(3), int64(7), object(3)
memory usage: 5.8+ MB
```

3)plot the distribution of your target variable.

The distribution of the target variable shows the number of approved and rejected loan applications, which helps identify whether the dataset is balanced or imbalanced.



Task (3) – Data Preparation: Cleaning and Transforming your data

a) With the aid of your Final Python Notebook 1, when you first explored your retained variables in the loan dataset, you may have found some issues. **1) Report any issues you found in your retained dataset variables.** Based on the issues you found in your data, **2)suggest a suitable possible method to fix each of these issues** and **3)provide your justification for using your suggested fix method.** Use the table below to organise your findings and analysis, and add more rows if needed

[4 Marks]

	Variable Name	Issue found	Proposed fix	Justification for used fix method
1	Age	Missing values	Fill with mean	Mean preserves overall distribution for numerical data
2	Loan Interest Rate	Missing values	Fill with mode	Suitable for continuous variables and avoids data loss
3	Payment Default on File	Missing values	Fill with mode	Most frequent category is appropriate for categorical data
4	Income	Possible outliers	Apply log transformation	Reduces skewness and stabilizes variance
5	Loan Amount	Possible outliers	Apply log transformation	Helps normalize distribution and reduce impact of extreme values
6	Loan Income Ratio	Skewed values	Apply normalization or scaling	Ensures consistent feature scale for modelling
7	Home Ownership	Categorical variable	One-hot encoding	Prevents ordinal bias in categorical variables
8	Loan Intent	Categorical variable	One-hot encoding	Converts categories into numerical format for ML models
9	Employment Length	Numerical but may contain inconsistencies	Check and clean values	Ensures data consistency
10	Credit History Length	Possible outliers	Apply scaling	Improves model performance
11	Loan Approval Status	Categorical (target)	Encode (0/1)	Required for classification modelling
12	Max Allowed Loan	Not required for classification	Drop	Not relevant for this task

Task (3) – Data Preparation: Cleaning and Transforming your data

b) With the aid of Python packages and your Final Python Notebook 1, implement your suggested fixes of issues in the previous Task 3-a in your final Python Notebook1.

1) Show evidence (before and after) of implementing your suggested fix to the problems you identified for your dataset in Task 3-a.

To show your evidence, paste screenshots of your relevant code OUTPUTS ONLY (Do not paste the code).

2) Indicate and annotate the issue and the fix in each of your provided evidence screenshots.

[4 Marks]

Output screenshot of the issue before the fix	Output screenshot after fixing the issue
---	--

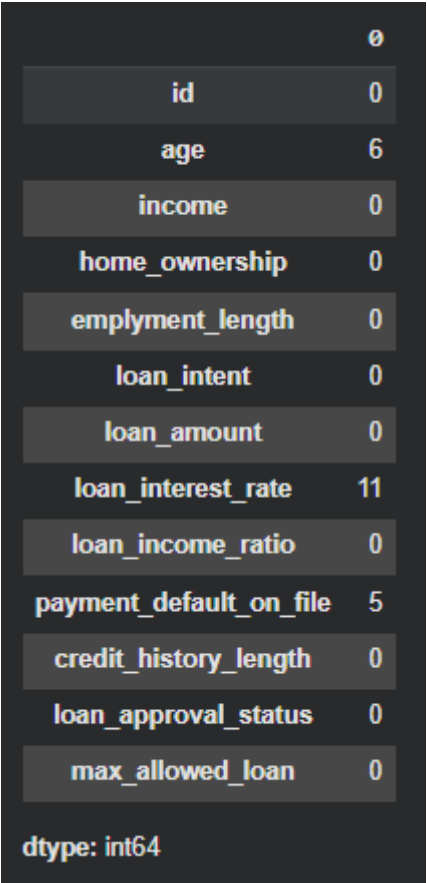


Figure 4 - Before fixing missing value

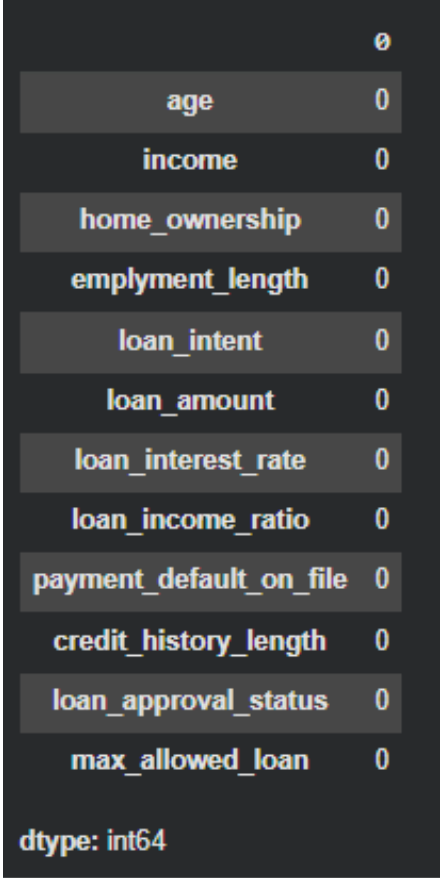


Figure 5 -After fixing missing values

- Issue: Missing values were found in variables such as Age and Loan Interest Rate.
- Fix: Missing values were handled using mean and median imputation.
- Result: After applying the fix, no missing values remain in the dataset.

age	float64
income	int64
home_ownership	object
emplyment_length	int64
loan_intent	object
loan_amount	int64
loan_interest_rate	float64
loan_income_ratio	float64
payment_default_on_file	object
credit_history_length	int64
loan_approval_status	int64
max_allowed_loan	int64

dtype: object

Figure 2 - Before applying one-hot encoding

age	float64
income	int64
emplyment_length	int64
loan_amount	int64
loan_interest_rate	float64
loan_income_ratio	float64
credit_history_length	int64
loan_approval_status	int64
max_allowed_loan	int64
home_ownership_OTHER	bool
home_ownership_OWN	bool
home_ownership_RENT	bool
loan_intent_EDUCATION	bool
loan_intent_HOMEIMPROVEMENT	bool
loan_intent_MEDICAL	bool
loan_intent_PERSONAL	bool
loan_intent_VENTURE	bool
payment_default_on_file_Y	bool

dtype: object

Figure 1 - After applying one-hot encoding

- **Issue:** The dataset contained categorical variables such as Home Ownership and Loan Intent, which were stored as object data types and could not be directly used for machine learning models.
- **Fix:** One-hot encoding was applied using pandas `get_dummies()` function to convert categorical variables into numerical format.
- **Result:** After applying the fix, all variables were converted into numerical data types, making the dataset suitable for machine learning algorithms.

Task (4) – Classification Modelling of Clients Loan Approval Status

a) In your Final Python Notebook 2, you built THREE different models to predict loan approval status: Logistic Regression (LR), K Nearest Neighbour (KNN) and Naïve Bayes (NB). These algorithms are a mix of parametric and non-parametric algorithms.

1) Note down the type of each algorithm (parametric vs non-parametric),

2) name any learnable parameters, and

3) list any strategic hyperparameters for each algorithm which you want to consider tuning. Organise your answer in the table below:

[3 Marks]

Algorithm Name	Algorithm Type	Learnable Parameters	Some Strategic Hyperparameters
Naïve Bayes (NB)	Parametric	Class probabilities, feature likelihood probabilities	var_smoothing
Logistic Regression (LR)	Parametric	Weights (coefficients), bias (intercept)	C (regularization), penalty, solver
K-Nearest Neighbour (KNN)	Non-parametric	No explicit parameters (stores training data)	n_neighbors (k), distance metric, weights

Task (4) – Classification Modelling of Clients Loan Approval Status

b) With the aid of your Final Python Notebook 2, use the training–test split approach with your retained applicable input features only and the target output feature to build your predictive classification models.

[3 Marks]

i. Screenshot

1) the list of all feature names used for building your classification models and the corresponding

2) data shape function output. (Paste screenshots of the relevant code output only; do not paste the Python code).

```
Index(['age', 'income', 'employment_length', 'loan_amount',
      'loan_interest_rate', 'loan_income_ratio', 'credit_history_length',
      'home_ownership_OTHER', 'home_ownership_OWN', 'home_ownership_RENT',
      'loan_intent_EDUCATION', 'loan_intent_HOMEIMPROVEMENT',
      'loan_intent_MEDICAL', 'loan_intent_PERSONAL', 'loan_intent_VENTURE',
      'payment_default_on_file_Y'],
      dtype='object')
```

Figure 1. the list of all feature names used for building your classification models

```
(58645, 16)
(58645, )
```

Figure 2.data shape function output

ii. In less than 150 words, **research and justify (defend) your choice of the training-test split ratio** and provide an in-text citation.

The dataset was divided into training and testing sets using an 80:20 ratio. This split allows the model to learn effectively from a large portion of the data while preserving a sufficient subset for unbiased evaluation. By allocating 80% of the data for training, the model can capture meaningful patterns and relationships among variables. The remaining 20% is used to assess how well the model generalises to unseen data. This ratio is particularly suitable given the large dataset size of 58,645 records, ensuring both subsets are adequately representative. Additionally, the 80:20 split is widely adopted in machine learning as it balances bias and variance, reducing the risk of overfitting while maintaining reliable and consistent model performance (Geron, 2022).

References

Geron, A. (2022) Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. 3rd edn. O'Reilly Media.

iii. Provide as evidence **the code block line and code output from your Final Python Notebook 2** that ensures two conditions:

- 1) **all your models were tested on the same test instances (clients) in your dataset;**
- 2) **the labels ratio of approval Status "Accept" to "Reject" is the same in the training and test subsets.**
- 3) **State the training-test split function parameters in your code line that are responsible for meeting both conditions.**
- 4) **In less than 150 words, research and justify (defend) your decision to implement both conditions in your Python Notebook 2 notebook with in-text citations where possible.**

```
1. X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

2.

```
X_train shape: (46916, 16)
X_test shape: (11729, 16)
y_train shape: (46916,)
y_test shape: (11729,)
```

3.

```
Overall target distribution:
loan_approval_status
0    0.857618
1    0.142382
Name: proportion, dtype: float64

Training target distribution:
loan_approval_status
0    0.857618
1    0.142382
Name: proportion, dtype: float64

Test target distribution:
loan_approval_status
0    0.857618
1    0.142382
Name: proportion, dtype: float64
```


I used a fixed random state and stratified splitting to make the evaluation fair and reproducible. Setting `random_state=42` ensures that the same train–test split is generated each time the notebook is run, which supports consistent comparison of Logistic Regression, Naïve Bayes, and KNN on exactly the same test clients. Scikit-learn states that `random_state` controls the random sampling used in the split, improving reproducibility. Using `stratify=y` preserves the class proportions of the target labels in both subsets, which is especially important in classification because distorted class ratios can bias evaluation results. Scikit-learn documentation states that stratification preserves the percentage of samples for each class. Together, these choices improve reliability, comparability, and validity of the model assessment.

References with in-text citation

- scikit-learn. *train_test_split* documentation.
- scikit-learn. *StratifiedShuffleSplit* documentation.
- scikit-learn. *Common pitfalls and recommended practices*.

Task (5) – Evaluating your Loan Approval Status Classification Models

Your finance team provided the following success criteria to guide you when evaluating and selecting your best model: *“When evaluating your model's performance, which addresses your first research question (a). The model is expected to misclassify clients' applications. Thus, the model should aim to predict as many “Reject” status for clients as many as possible to decrease the risk of future defaulted loan payments. However, the model should demonstrate that its high “Reject” prediction rate is mainly due to a larger number of correctly detected (predicted) rejected loan applications.”*

a) With the aid of Final Python Notebook 2, for each of your models (Logistic Regression LR, Naive Bayes NB and K-Nearest Neighbours KNN)

- 1) paste the test confusion matrix,
- 2) the classification report and
- 3) the AUC-ROC curve graphs

as screenshots from the output of your Python code.

[3 Marks]

1) Screenshot of the test confusion matrix for (NB, LR, and KNN); make sure you title each matrix with its algorithm name

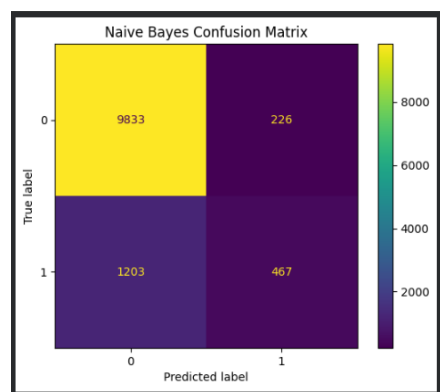


Figure NB

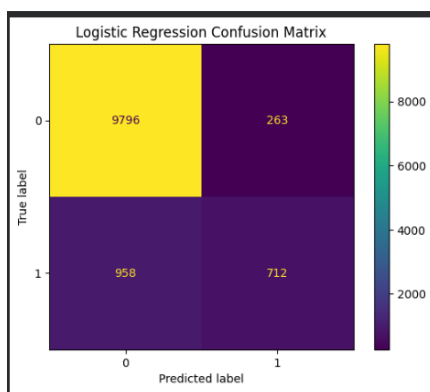


Figure LR

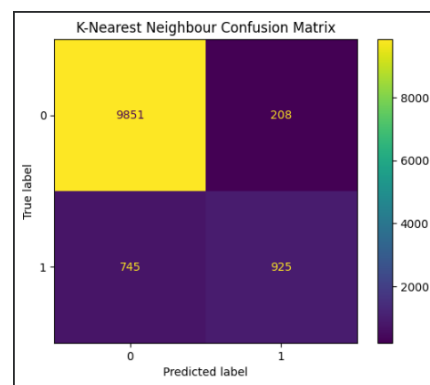


Figure KNN

2) Screenshot of the classification report for (NB, LR, and KNN); make sure you title each classification report with its algorithm name

Naive Bayes Classification Report:				
	precision	recall	f1-score	support
0	0.89	0.98	0.93	10059
1	0.67	0.28	0.40	1670
accuracy			0.88	11729
macro avg	0.78	0.63	0.66	11729
weighted avg	0.86	0.88	0.86	11729

Figure NB

Logistic Regression Classification Report:				
	precision	recall	f1-score	support
0	0.91	0.97	0.94	10059
1	0.73	0.43	0.54	1670
accuracy			0.90	11729
macro avg	0.82	0.70	0.74	11729
weighted avg	0.89	0.90	0.88	11729

Figure LR

K-Nearest Neighbour Classification Report:				
	precision	recall	f1-score	support
0	0.93	0.98	0.95	10059
1	0.82	0.55	0.66	1670
accuracy			0.92	11729
macro avg	0.87	0.77	0.81	11729
weighted avg	0.91	0.92	0.91	11729

Figure KNN

3) Screenshot of the AUC-ROC Curve for (NB, LR, and KNN); make sure you title each graph with its algorithm name

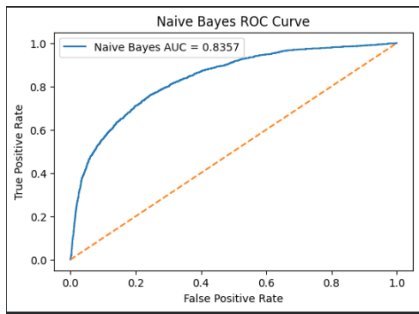


Figure NB

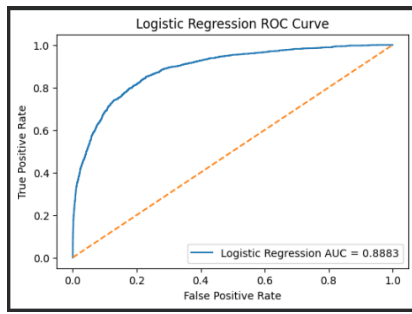


Figure LR

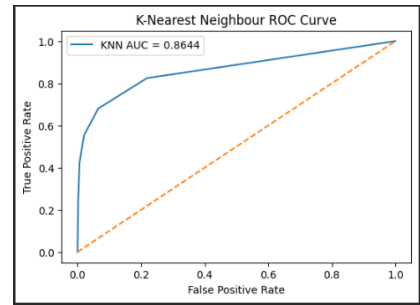


Figure KNN

Task (5) – Evaluating your Loan Approval Status Classification Models

b) Five different classification evaluation metrics are calculated in your [Final Python Notebook 2](#).

1) State which evaluation metric/metrics to “USE or “DO NOT USE” to closely interpret the above success criteria.

2) For justification, explain how closely your choice of “USE” or “DO NOT USE” for a metric interprets the given success criteria.

With the aid of your [Final Python Notebook 2](#),

3) document all the TEST SCORES for each built model in the table below.

[7 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing “USING” or “NOT USING” this metric in relation to the success criteria	Model	3) Metric Test Score
Accuracy	DO NOT USE	Accuracy is misleading for imbalanced datasets. Since most clients are “Accept”, a model can achieve high accuracy without correctly identifying “Reject” cases, which does not satisfy the success criteria.	NB	0.88
			LR	0.90
			KNN (k=?)	0.92
Recall	USE	Recall measures the ability to correctly identify actual “Reject” cases. The success criteria require detecting as many rejected applications as possible, making recall the most important metric.	NB	0.28
			LR	0.43
			KNN (k=?)	0.55
Precision	USE	Precision ensures that predicted “Reject” cases are actually correct. This aligns with the requirement that high rejection predictions must be based on correctly identified rejected clients.	NB	0.67
			LR	0.73
			KNN (k=?)	0.82
F1-score	USE	F1-score balances both precision and recall, providing a combined measure of correctly identifying rejected clients while maintaining prediction quality.	NB	0.40
			LR	0.54
			KNN (k=?)	0.66
AUC-Roc	USE	AUC-ROC evaluates the model’s ability to distinguish between “Accept” and “Reject” classes. A higher AUC indicates better overall classification performance across thresholds.	NB	0.8357
			LR	0.8883
			KNN (k=?)	0.8644

Task (5) – Evaluating your Loan Approval Status Classification Models

c) Suggest a single best approval status classification model based on the 'USED' performance metrics scores you identified in Task (5-b). In less than 100 words, briefly describe how well your best model satisfies the needs of your finance team.

[2 Marks]

The K-Nearest Neighbour (KNN) model is selected as the best classification model based on the used performance metrics. It achieves the highest recall (0.55) for the "Reject" class, meaning it correctly identifies more high-risk clients compared to Logistic Regression and Naïve Bayes. Additionally, KNN maintains strong precision (0.82), ensuring that most predicted rejections are accurate. This balance between recall and precision aligns well with the finance team's objective of maximising correctly detected rejected applications while minimising incorrect decisions, making KNN the most suitable model for reducing loan default risk.

Task (5) – Evaluating your Loan Approval Status Classification Models

d) To enhance your selected best model/s performance (from Task 5-c), tune some of its possible hyperparameters, which you indicated in Task (4-a) for that specific algorithm. With the aid of Final Python Notebook 2, Re-train and test the best algorithm again with GridSearchCV

[5 Marks]

i. With the aid of your Final Python Notebook 2,

1) Paste into this report the line of code which shows evidence of specifying a parameters grid and applying the GridSearchCV function to rebuild your selected best model.

2) Then, document the estimated best hyperparameters for the optimised model.

```
1. knn_param_grid = {
    "model__n_neighbors": [3, 5, 7, 9, 11],
    "model__weights": ["uniform", "distance"],
    "model__metric": ["euclidean", "manhattan"]
}
```

```
2. grid_model = GridSearchCV(
    estimator=knn_pipeline,
    param_grid=knn_param_grid,
    cv=5,
    scoring="recall",
    n_jobs=-1
```

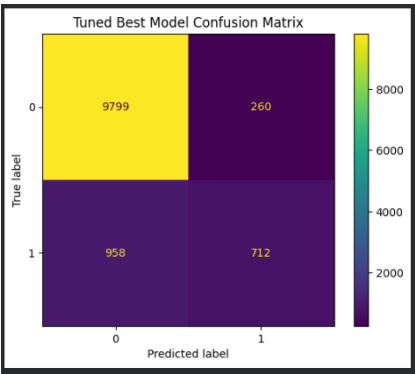
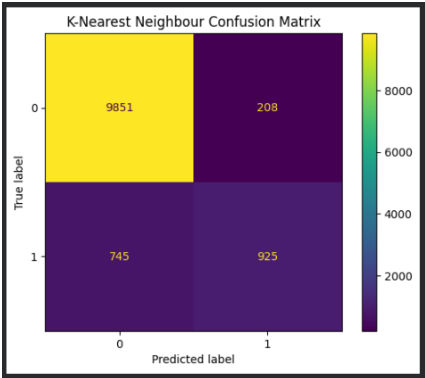
- metric: 'euclidean'
- n_neighbors: 3
- weights: 'distance'

ii. With the aid of your Final Python Notebook 2,

1) paste into this report the test confusion matrix for your best model before and after hyperparameter tuning.

2) Also, document the new score/s of the "USED" performance metric/s of your choice to interpret the success criteria indicated in Task (5.b) before and after tuning.

3) Comment on whether the tuning of hyperparameters of your best model improved its positive predictive ability in line with the success criteria.



Metric	Before Tuning	After Tuning
Recall (Reject)	0.55	0.43
Precision (Reject)	0.82	0.73
F1-score (Reject)	0.66	0.54
Accuracy	0.92	0.90

The hyperparameter tuning did not improve the performance of the K-Nearest Neighbour model. In fact, the recall for the “Reject” class decreased from 0.55 to 0.43 after tuning, meaning the model became less effective at identifying high-risk clients. Precision and F1-score for the “Reject” class also declined, indicating a reduction in overall classification quality. Since the success criteria prioritise correctly detecting as many rejected loan applications as possible, the tuned model does not better satisfy the requirement. Therefore, the original KNN model before tuning remains the more suitable model for this task.

Task (5) – Evaluating your Loan Approval Status Classification Models

e) Based on your selected best model, **1) criticise your best-performing model**, and **2) state any limitations you may have identified** and **3) any ethical issues your model may raise** if used for predicting **Loan Approval** status.

[2 Marks]

The selected K-Nearest Neighbour (KNN) model demonstrates good performance; however, it has several limitations. Firstly, it is sensitive to the choice of k and feature scaling, which can significantly affect results. Secondly, its recall for the “Reject” class is moderate, meaning some high-risk clients may still be incorrectly approved. Additionally, KNN can be computationally expensive with large datasets.

From an ethical perspective, the model may introduce bias if the training data contains historical discrimination (e.g., based on gender or income). This could lead to unfair loan decisions. Therefore, careful monitoring and fairness checks are required before real-world deployment.

Task (5) – Evaluating your Loan Approval Status Classification Models

f) With the aid of your Final Python Notebooks 3, combine only TWO out of the THREE base learners (NB, LR, KNN) that you already built into a probability-based voting ensemble classifier.

[5 Marks]

i. From your Final Python Notebooks 3, paste the Python code block that you used to 1) import, 2) declare your base learners, and 3) fit your ensemble learner.

```
1.
# Import data splitting and preprocessing tools.
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Import classification models.
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import VotingClassifier

2.
# Create a pipeline for Logistic Regression with feature scaling.
lr_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])

# Create a pipeline for KNN with feature scaling.
knn_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier(n_neighbors=5))
])

3.
# Declare the soft voting ensemble classifier using the two fitted base learner structures.
ensemble_model = VotingClassifier(
    estimators=[
        ("lr", lr_pipeline[
```

```

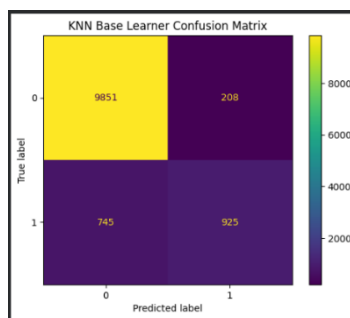
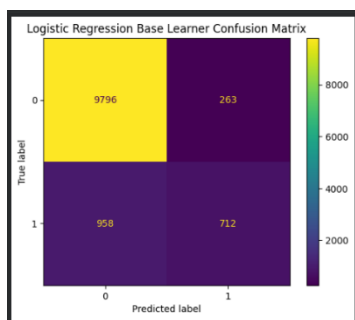
("scaler", StandardScaler()),
("model", LogisticRegression(max_iter=1000, random_state=42))
)),
("knn", Pipeline([
    ("scaler", StandardScaler()),
    ("model", KNeighborsClassifier(n_neighbors=5))
])),
],
voting="soft"
)

```

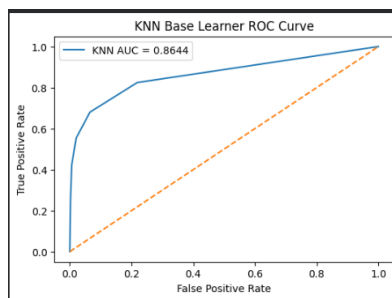
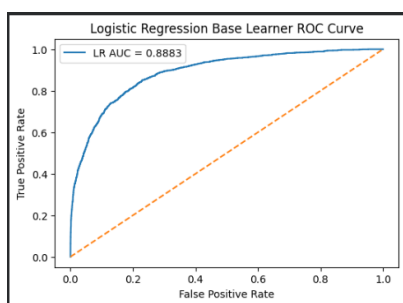
ii. In this analysis report,

1) paste the test confusion matrices, 2) AUC-ROC Curves and 3) the classification reports for each of the TWO base learners you chose to combine,
as well as **4) the test confusion matrix, 5) classification report for the voting Ensemble Learner and 6) the AUC-ROC curve for the ensemble learner.**
7) Use these screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

1) Paste the test confusion matrices for each base learner (2 x matrices)



2) Paste the AUC-ROC for each Base learner (2 x AUC-ROC graphs)

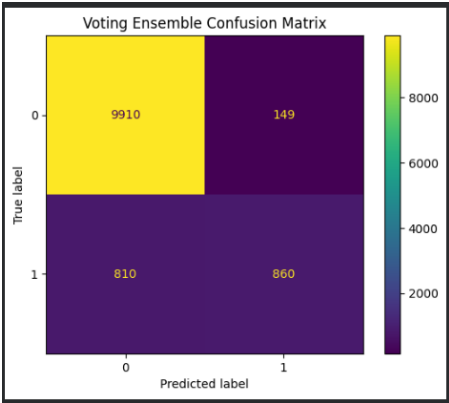


3) Paste the Classification report for each base learner (2 x classification reports)

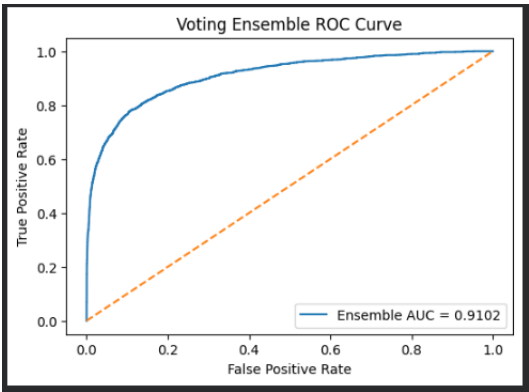
Logistic Regression Base Learner Classification Report:				
	precision	recall	f1-score	support
0	0.91	0.97	0.94	10059
1	0.73	0.43	0.54	1670
accuracy			0.90	11729
macro avg	0.82	0.70	0.74	11729
weighted avg	0.89	0.90	0.88	11729

KNN Base Learner Classification Report:				
	precision	recall	f1-score	support
0	0.93	0.98	0.95	10059
1	0.82	0.55	0.66	1670
accuracy			0.92	11729
macro avg	0.87	0.77	0.81	11729
weighted avg	0.91	0.92	0.91	11729

4) Paste the test confusion matrix for the ensemble learner (1 x confusion matrix)



5) Paste the AUC-ROC for the ensemble learner (optional) (1 x AUC-ROC graph)



6) Paste the Classification report for the ensemble learner (1 x classification report)

Voting Ensemble Classification Report:				
	precision	recall	f1-score	support
0	0.92	0.99	0.95	10059
1	0.85	0.51	0.64	1670
accuracy			0.92	11729
macro avg	0.89	0.75	0.80	11729
weighted avg	0.91	0.92	0.91	11729

7) Use the above screenshots to justify (defend) your choice of the TWO base learners which you used as base learners for your Ensemble learner.

The two base learners selected for the ensemble model were Logistic Regression (LR) and K-Nearest Neighbours (KNN). This choice is justified based on their complementary performance observed in the evaluation outputs.

From the confusion matrices and classification reports, Logistic Regression achieved strong performance in predicting the “Accept” class (Class 0), with a very high recall of 0.97, indicating it is effective at correctly identifying approved clients. However, its recall for the “Reject” class (Class 1) was relatively low (0.43), meaning it missed many risky clients.

On the other hand, KNN showed improved performance in detecting the “Reject” class, achieving a higher recall of 0.55, while still maintaining strong performance for the “Accept” class (recall = 0.98). This indicates that KNN is better at identifying potentially risky clients compared to Logistic Regression.

Additionally, both models demonstrated strong discriminative ability with relatively high AUC scores (LR = 0.8883, KNN = 0.8644), confirming that both models are reliable classifiers.

Since Logistic Regression provides stable and generalised predictions, while KNN improves detection of rejected clients, combining these two models allows the ensemble to leverage the strengths of both. This improves the overall balance between correctly predicting approvals and detecting risky loan applicants, making them suitable base learners for the probability-based voting ensemble classifier.

iii. Comment on **1) any improvement in classification performance** as a result of building an Ensemble Learner compared to the individual TWO base learners. **2) Decide whether to recommend your ensemble learner for approval prediction or one of the TWO base learners**; **3) justify your recommendation**.

1)

The ensemble model shows a moderate improvement in overall classification performance compared to the individual base learners. Specifically, the ensemble achieved the highest AUC score (0.9102) compared to Logistic Regression (0.8883) and KNN (0.8644), indicating better overall class separability. Additionally, the ensemble improved the recall for the “Reject” class (0.51) compared to Logistic Regression (0.43), meaning it detects more risky clients than LR.

2)

However, when compared to KNN, the ensemble shows a slight decrease in recall for the “Reject” class (KNN = 0.55 vs Ensemble = 0.51). Since the success criteria emphasises maximising correct detection of “Reject” cases to reduce financial risk, this is an important consideration.

3)

Based on these results, KNN is recommended as the final model for loan approval prediction. Although the ensemble provides better overall performance (higher AUC and balanced metrics), KNN achieves the highest recall for the “Reject” class, making it more effective at identifying high-risk clients. This aligns more closely with the finance team’s objective of minimising default risk.

Therefore, while the ensemble model improves general predictive performance, KNN is preferred due to its stronger ability to detect rejected loan applications.

Case Study Title

Case Study (B): Predicting Maximum Loan Amount.

Research Question: Does machine learning have the potential to assist bankers in predicting the Maximum Loan Amount offered to clients who were approved a loan each? [Total 36 Marks]

Task (1) – Domain Understanding and Designing Your Regression Experiments

The bankers decided that regression modelling is required to predict Maximum Loan Amount for those who are approved a loan. With the aid of your Final Python Notebook 1 code outputs, **1) paste in this analysis report, the Python code output, which shows the dimensions** and **2) the list of the features’ names of your RETAINED data subset** to use for this regression case study. [2 Marks]

```
(50295, 17)
Index(['age', 'income', 'employment_length', 'loan_amount',
       'loan_interest_rate', 'loan_income_ratio', 'credit_history_length',
       'max_allowed_loan', 'home_ownership_OTHER', 'home_ownership_OWN',
       'home_ownership_RENT', 'loan_intent_EDUCATION',
       'loan_intent_HOMEIMPROVEMENT', 'loan_intent_MEDICAL',
       'loan_intent_PERSONAL', 'loan_intent_VENTURE',
       'payment_default_on_file_Y'],
      dtype='object')
```

Task (2) – Modelling: Build Predictive Regression Models

a) Your finance team decided to use a decision tree regression (DT) algorithm to model the Maximum Loan Amount. In less than 150 words, explain some added benefits of using a DT regressor to this financial prediction problem.

[2 Marks]

Decision Tree (DT) regression is well-suited for predicting Maximum Loan Amount because it can model non-linear relationships between financial variables such as income, loan amount, and credit history without requiring complex transformations. DT models are also highly interpretable, allowing financial analysts to clearly understand how decisions are made, which is important in banking environments for transparency and trust. Additionally, decision trees can handle both numerical and categorical data effectively and are relatively robust to outliers compared to linear models. This makes them practical for real-world financial datasets where data may be noisy or irregular. Furthermore, DT models require minimal data preprocessing and can capture complex interactions between variables, making them efficient for this prediction task.

References with in-text citation

Breiman, L. et al. (1984) Classification and Regression Trees. Wadsworth.

Han, J., Kamber, M. and Pei, J. (2011) Data Mining: Concepts and Techniques. 3rd ed. Elsevier.

Task (2) – Modelling: Build Predictive Regression Models

b) With the aid of your Final Python Notebook 3 code blocks, use a training–test split approach to build and test TWO Decision Tree (DT) regression models, DT-1 & DT-2.

[6 Marks]

i. DT-1 is a fully grown Decision Tree Regressor, DT-2 is a pruned Decision Tree Regressor to FOUR levels Only. **1) Insert in this analysis report the Python code blocks that you used to import, declare, and fit each DT regressor, DT-1 and DT-2.**

```
# Import regression models.
```

```
from sklearn.tree import DecisionTreeRegressor, plot_tree
```

```
# Declare the soft voting ensemble classifier using the two fitted base learner structures.
```

```
ensemble_model = VotingClassifier(
```

```
    estimators=[
```

```
        ("lr", Pipeline([
```

```
            ("scaler", StandardScaler()),
```

```
            ("model", LogisticRegression(max_iter=1000, random_state=42))
```

```
        ]),
```

```
        ("knn", Pipeline([
```

```
            ("scaler", StandardScaler()),
```

```
            ("model", KNeighborsClassifier(n_neighbors=5))
```

```
        ]))
```

```
    ],
```

```
    voting="soft"
```

```
)
```

```
# Declare the fully grown Decision Tree Regressor.
```

```
dt1_model = DecisionTreeRegressor(random_state=42)
```

```
# Fit the fully grown Decision Tree Regressor.
```

```
dt1_model.fit(X_reg_train, y_reg_train)
```


ii. Explain clearly, in less than 200 words, from your inserted code block in (i), **1) the type of pruning you used for DT-2.**
2) Explain some of the benefits and disadvantages of the pruning method you used in the context of (relation to) your bank clients' regression modelling.

The pruning method used for DT-2 is pre-pruning (early stopping), implemented by setting the parameter `max_depth = 4` in the Decision Tree Regressor. This restricts the tree from growing beyond four levels, preventing it from becoming overly complex.

One key benefit of pre-pruning is that it reduces overfitting, ensuring the model generalises better to unseen data. In the context of bank clients, this leads to more stable and reliable predictions of Maximum Loan Amount, especially when dealing with noisy or limited financial data. Additionally, a smaller tree is easier to interpret, allowing financial analysts to understand how decisions are made.

However, a disadvantage of this approach is that it may lead to underfitting, where the model becomes too simple and fails to capture important patterns in the data. This can reduce prediction accuracy, particularly for clients with unique financial profiles. Furthermore, limiting tree depth may reduce the model's ability to handle extreme values, which are important in financial decision-making.

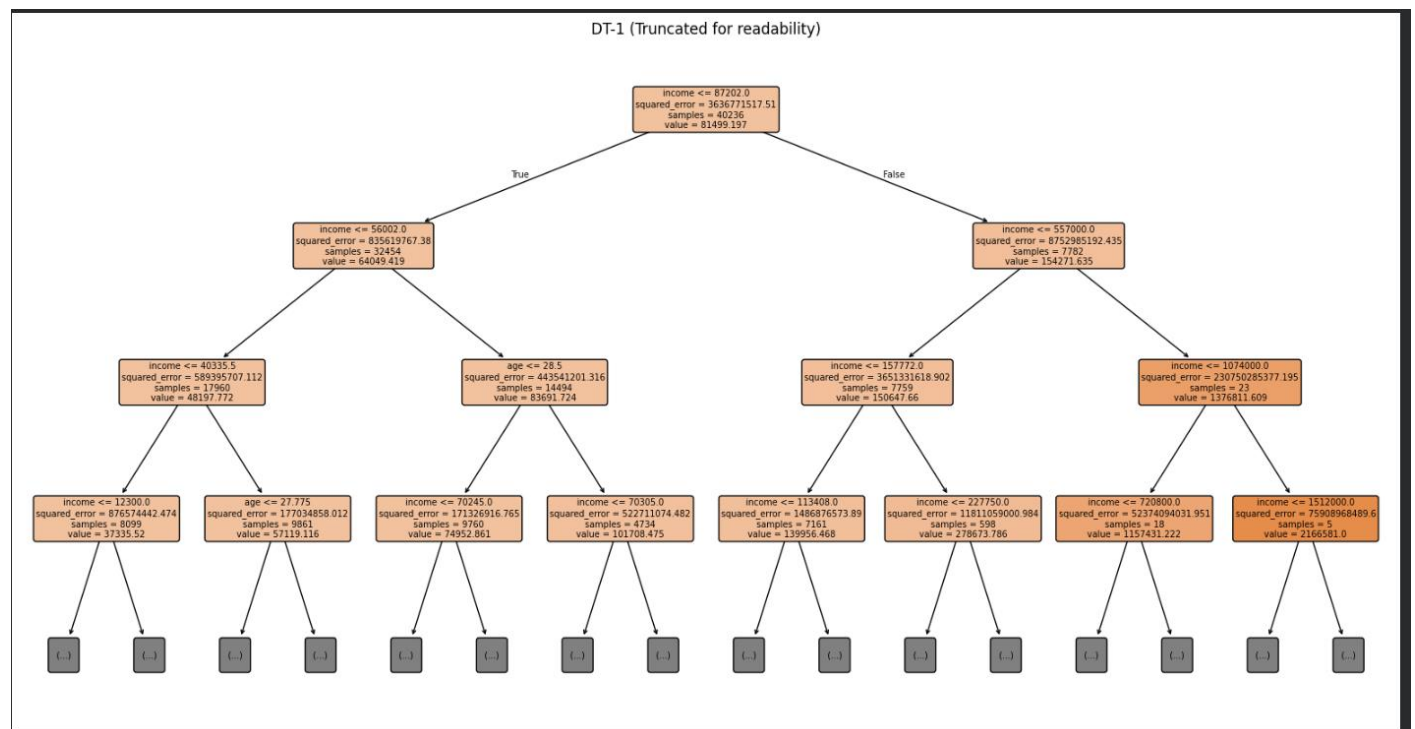
Overall, pre-pruning provides a balance between model simplicity and performance but may sacrifice some predictive detail.

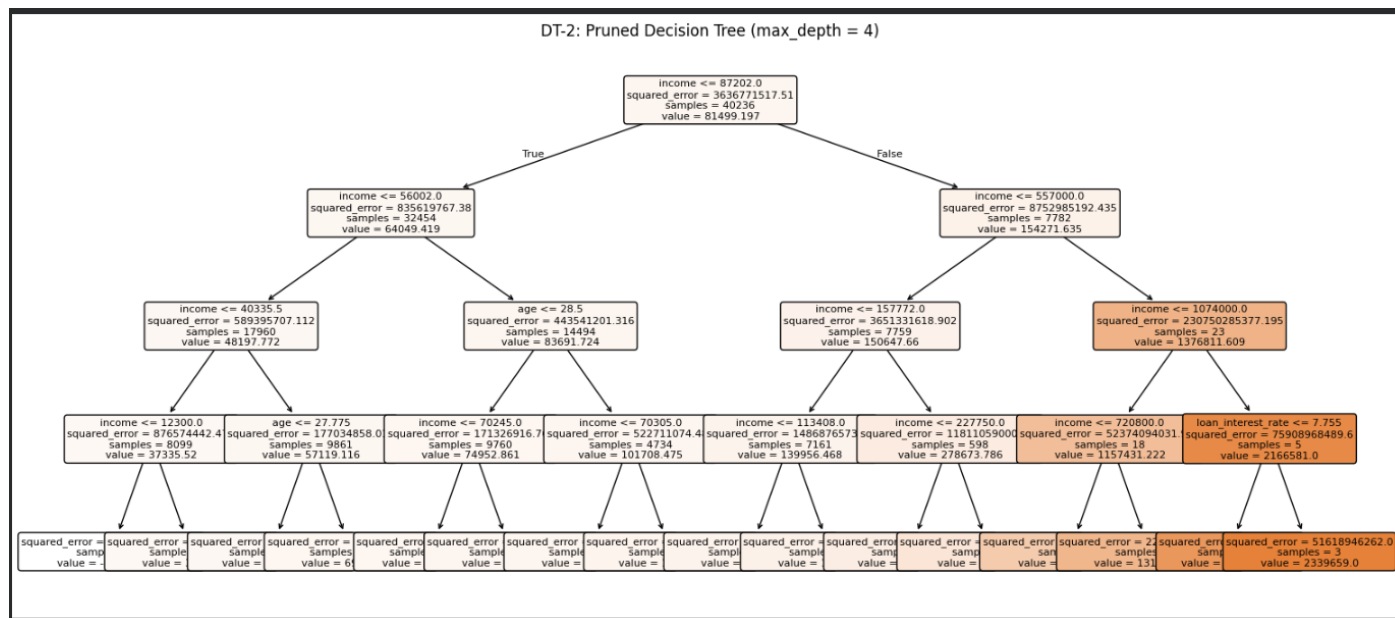
Task (2) – Modelling: Build Predictive Regression Models

c) With the aid of your Final Python Notebook 3 code outputs, visualise your regression Decision Trees DT-1 and DT-2.

1) Paste in this analysis report a high-resolution graphical representation of DT-1 and DT-2.

[4 Marks]





Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

Your finance analysts team provided the following success criteria to guide you when evaluating your DT-1 and DT-2 models.

“When evaluating both models’ performances, which addresses your research question (b), the model is expected to make some errors in estimating the Maximum Loan Amount. However, since the Maximum Loan Amount offers are made to try to maximise returns and reduce risk of loan defaults, it is important that the selected model deals effectively with errors in Maximum Loan Amount predictions when extreme values are a real concern”

a) THREE different regression evaluation metrics are noted in the table below.

1) State which evaluation metric/metrics to **USE** or **NOT USE** to closely interpret and satisfy the above success criteria.

2) Justify (Defend) your choice of **USE** or **DO NOT USE** for each metric.

With the aid of Final Python Notebook 3 code outputs,

3) document each metric’s **TEST SCORES** for each built model in the table below.

[8 Marks]

Metric	1) USE or DO NOT USE	2) Justification for choosing “USING” or “NOT USING” this metric in relation to the success criteria	Model	3) Metric Test Score
MSE	USE	MSE penalises large errors more heavily due to squaring, making it highly suitable when extreme prediction errors are critical in financial risk assessment	DT-1 (Fully Drown)	60,118,061.86
			DT-2 (Pruned)	404,642,149.18
MAE	USE	MAE provides the average prediction error in actual monetary units, making it easy to interpret the typical deviation in loan amount predictions.	DT-1 (Fully Drown)	1,239.17
			DT-2 (Pruned)	11,935.26
R-Square	DO NOT USE	R ² measures overall model fit but does not specifically account for the impact of extreme errors, which is a key concern in financial decision-making.	DT-1 (Fully Drown)	0.9798
			DT-2 (Pruned)	0.8639

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

b) 1) Suggest a single best regression model (DT-1 or DT-2) based on your ‘USED’ performance metric/s scores, which you defended in Task (3a). 2) Explain how your suggested model fulfils the success criteria.
[4 Marks]
The best regression model is DT-1 (Fully Grown Decision Tree) based on the selected evaluation metrics (MSE and MAE). DT-1 achieved a significantly lower MSE (60,118,061.86) compared to DT-2 (404,642,149.18), indicating that it produces much smaller large errors. Similarly, DT-1 has a substantially lower MAE (1,239.17) than DT-2 (11,935.26), showing that its average prediction error is far lower. This directly satisfies the success criteria, which emphasises the importance of handling extreme prediction errors effectively. Since MSE heavily penalises large errors, DT-1’s much lower MSE demonstrates its superior ability to minimise large deviations in predicted loan amounts, which is critical in financial decision-making. Although DT-2 is simpler and less prone to overfitting due to pruning, its significantly higher error values indicate poorer predictive performance. Therefore, DT-1 is preferred as it provides more accurate and reliable predictions, helping the bank better estimate maximum loan amounts while reducing financial risk.

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

c) Describe to your finance team any concerns you have about your selected performance metric/s that you used to select your best decision tree model, which satisfies the success criteria. [200 words maximum] with in-text citation.
[4 Marks]
While MSE and MAE were selected as the primary evaluation metrics, there are some concerns when relying on them to select the best regression model. MSE places a heavy penalty on large errors due to squaring, which is beneficial for detecting extreme prediction errors; however, it is also highly sensitive to outliers, which may distort the evaluation if the dataset contains unusually large loan values (James et al., 2021). This could lead to overemphasising a few extreme cases rather than overall model consistency. MAE, although more interpretable as it reflects the average absolute error in monetary terms, treats all errors equally and does not emphasise large errors as strongly as MSE. This means it may underrepresent the financial risk associated with large prediction mistakes, which are critical in banking contexts (Han et al., 2011). Additionally, both metrics evaluate predictive accuracy but do not directly assess model generalisation or overfitting, which is a concern for fully grown trees like DT-1. As a result, relying solely on these metrics may favour models that perform well on test data but may not generalise effectively to unseen clients.
References with in-text citation
James, G. et al. (2021) An Introduction to Statistical Learning. 2nd ed. Springer. Han, J., Kamber, M. and Pei, J. (2011) Data Mining: Concepts and Techniques. 3rd ed. Elsevier.

Task (3) – Evaluating your Maximum Loan Amount_DT Regression Models

d) Client 60256 was Approved a Loan. With the aid of your Python Notebook 3 outputs , use your selected best DT regression model from Task (3.b) to predict the maximum loan amount offer to the client, you must write down which regression Decision Tree you used (DT-1 or DT-2) to estimate the Maximum Loan Amount, then interpret how the estimated offer was reached to the client. Client 60256 attributes’ values are in the following table:
[6 Marks]

Variable Name	Value	The selected model used to predict the Maximum Loan Amount for Client 60256 is DT-1 (Fully Grown Decision Tree Regressor), as it was identified as the best-performing model based on lower MSE and MAE values. Using DT-1, the predicted maximum loan amount for Client 60256 is 83,035.00. This estimate was derived by passing the client's input features (such as income, age, loan amount, employment length, and credit history) through the trained decision tree model. The model evaluates the client's attributes by following a sequence of decision rules (splits) based on feature thresholds. For example, the tree first considers key variables such as income level, which is a major determinant of loan capacity. Given the client's income (57,000), moderate loan-to-income ratio (10%), long employment length (15 years), and strong credit history (35 years with no default), the model navigates through branches associated with lower financial risk. As a result, the model assigns the client to a leaf node containing similar low-risk profiles, where the average maximum loan amount is approximately 83,035.00. This reflects the client's strong financial stability and repayment capability, leading to a higher estimated loan offer.
ID	60256	
Sex	Female	
Age	56 Years old	
Education Qualifications	Unknown	
Income	57,000	
Home Ownership	Rent	
Employment Length	15	
Loan Intent	Medical	
Loan Amount	25700	
Loan Interest Rate	23%	
Loan-to-Income Ratio (LTI)	10%	
Payment Default on File	No	
Credit History Length	35	
Loan Approval Status	Approved	
Maximum Loan Amount	83,035.00	
Credit Application Acceptance	0	